Both DTOs

TS create-task.dto.ts U

src > tasks > dto > TS create-task.dto.ts > ...

```ts
import { IsNotEmpty, IsString } from 'class-validator';

export class CreateTaskDto {
  @IsString()
  @IsNotEmpty()
  title!: string;
}

```

TS update-task.dto.ts U

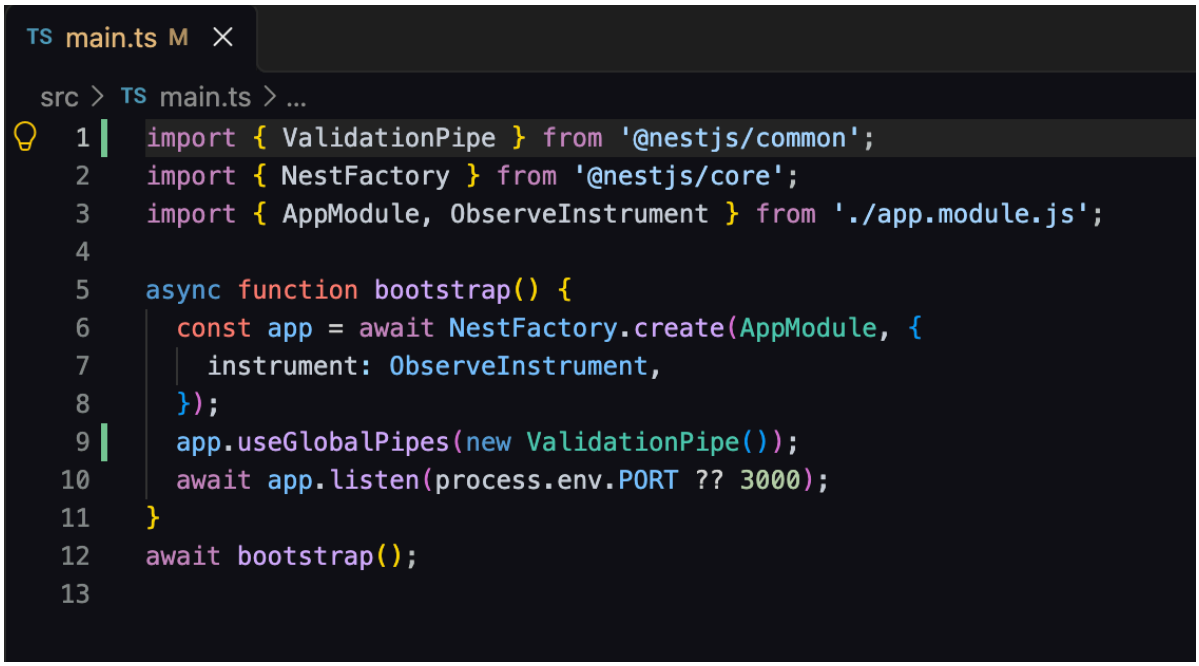
src > tasks > dto > TS update-task.dto.ts > ...

```ts
import { IsBoolean } from 'class-validator';

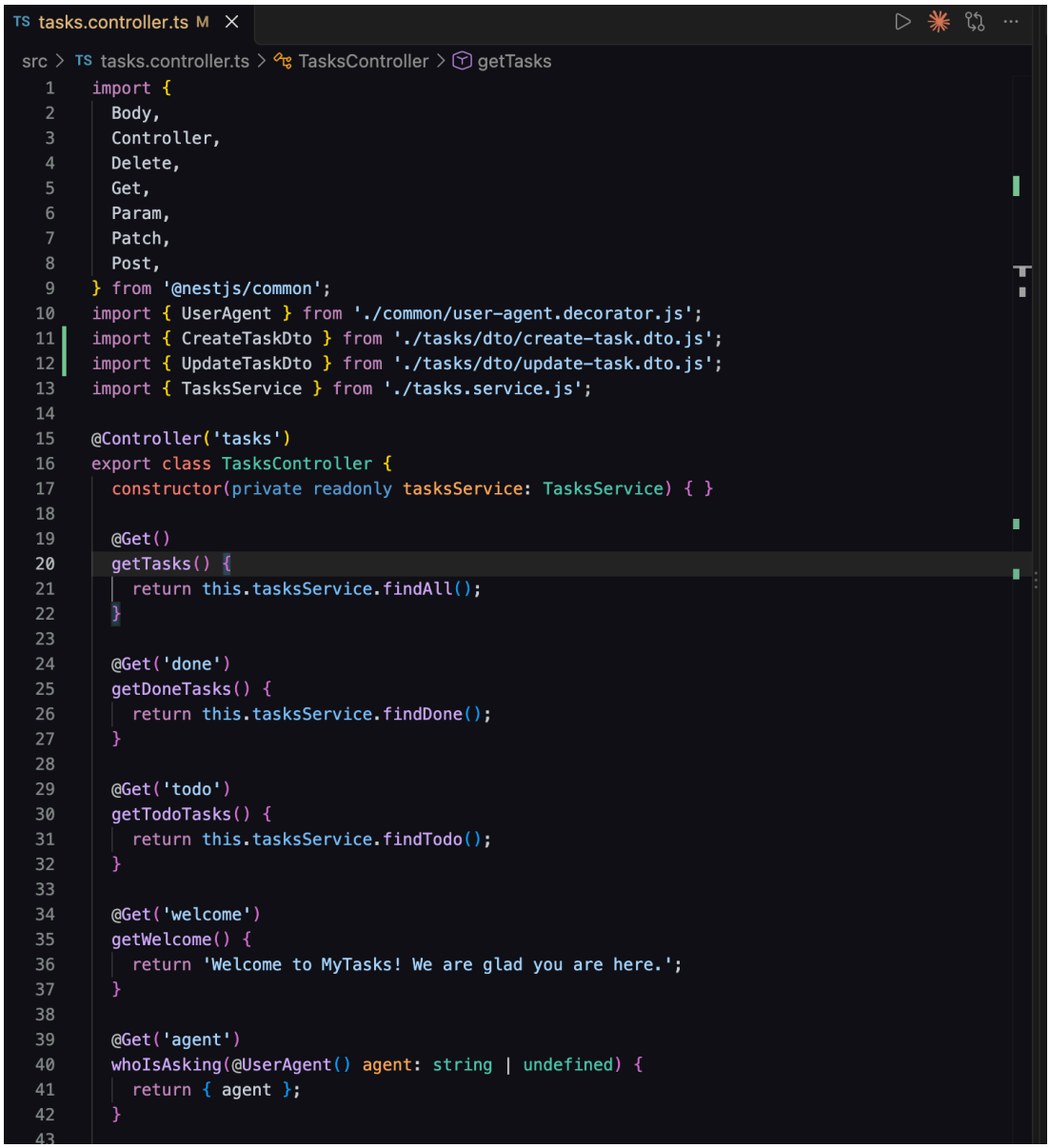
export class UpdateTaskDto {
  @IsBoolean()
  done!: boolean;
}

```

main.ts


TS main.ts M
src > TS main.ts > ...
1 import { ValidationPipe } from '@nestjs/common';
2 import { NestFactory } from '@nestjs/core';
3 import { AppModule, ObserveInstrument } from './app.module.js';
4
5 async function bootstrap() {
6 const app = await NestFactory.create(AppModule, {
7 instrument: ObserveInstrument,
8 });
9 app.useGlobalPipes(new ValidationPipe());
10 await app.listen(process.env.PORT ?? 3000);
11 }
12 await bootstrap();
13

tasks.controller.ts



TS tasks.controller.ts M

~/Desktop/Techpreneur/NestJS/mytasks/src/tasks.controller.ts • Modified

```
  }

  @Post()
  createTask(@Body() body: CreateTaskDto) {
    return this.tasksService.create(body.title);
  }

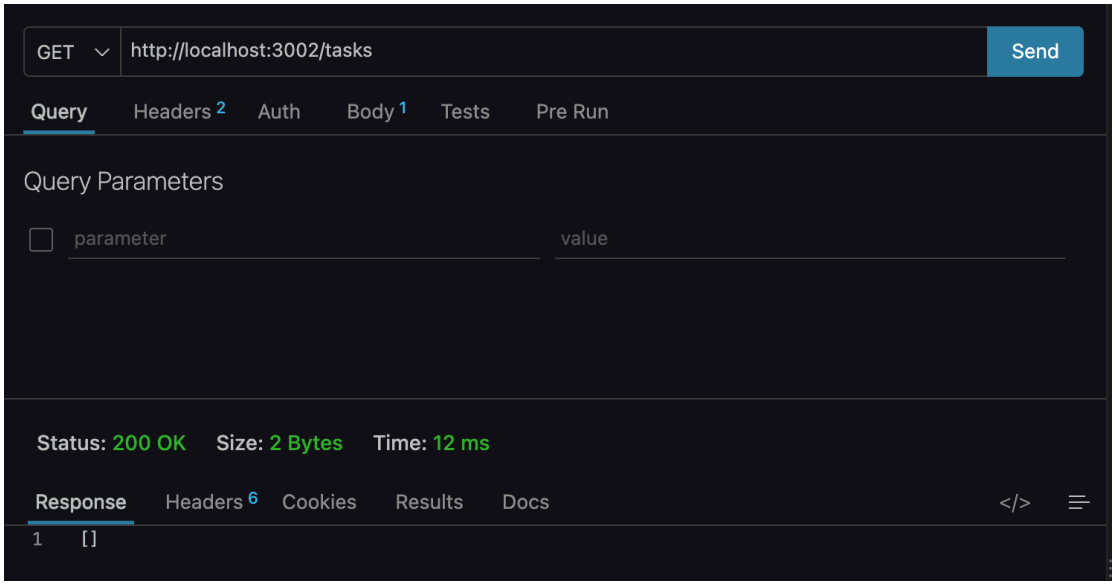
  @Patch(':id')
  updateTask(@Param('id') id: string, @Body() body: UpdateTaskDto) {
    return this.tasksService.update(Number(id), body.done);
  }

  @Delete(':id')
  deleteTask(@Param('id') id: string) {
    return this.tasksService.remove(Number(id));
  }

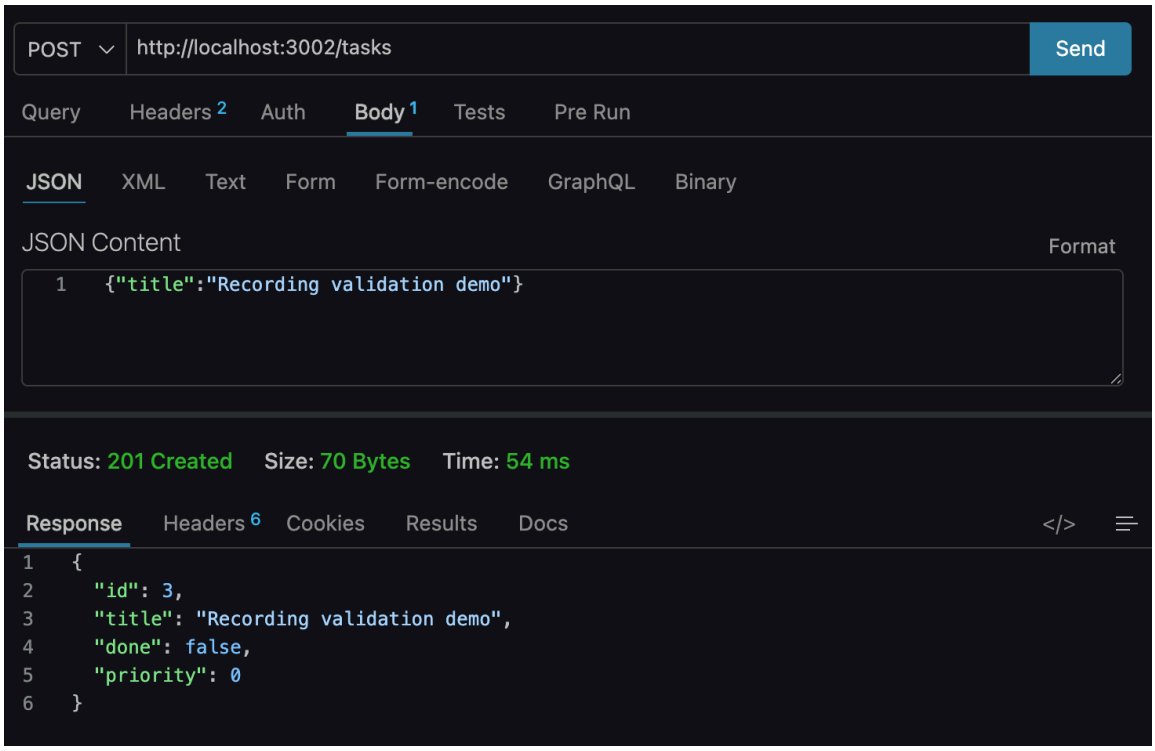
  @Get(':id')
  async getTaskById(@Param('id') id: string) {
    const task = await this.tasksService.findOne(Number(id));
    return task;
  }
}

```

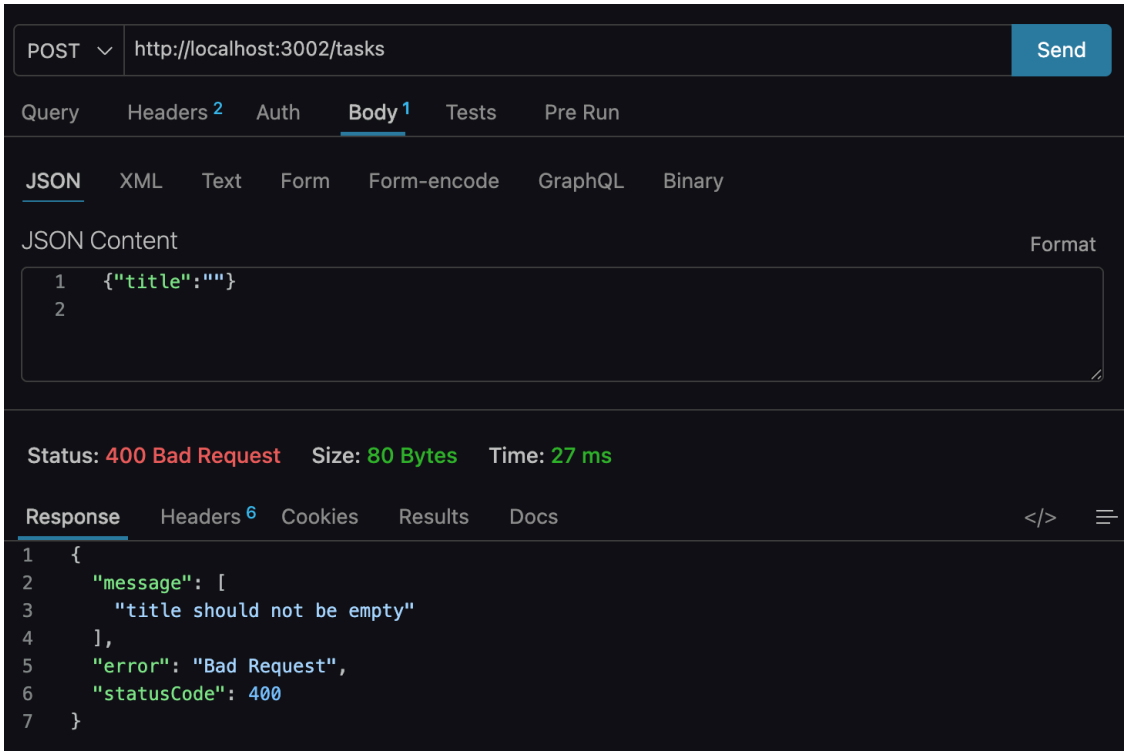
Test six cases in Thunder Client and show each response in the screenshot



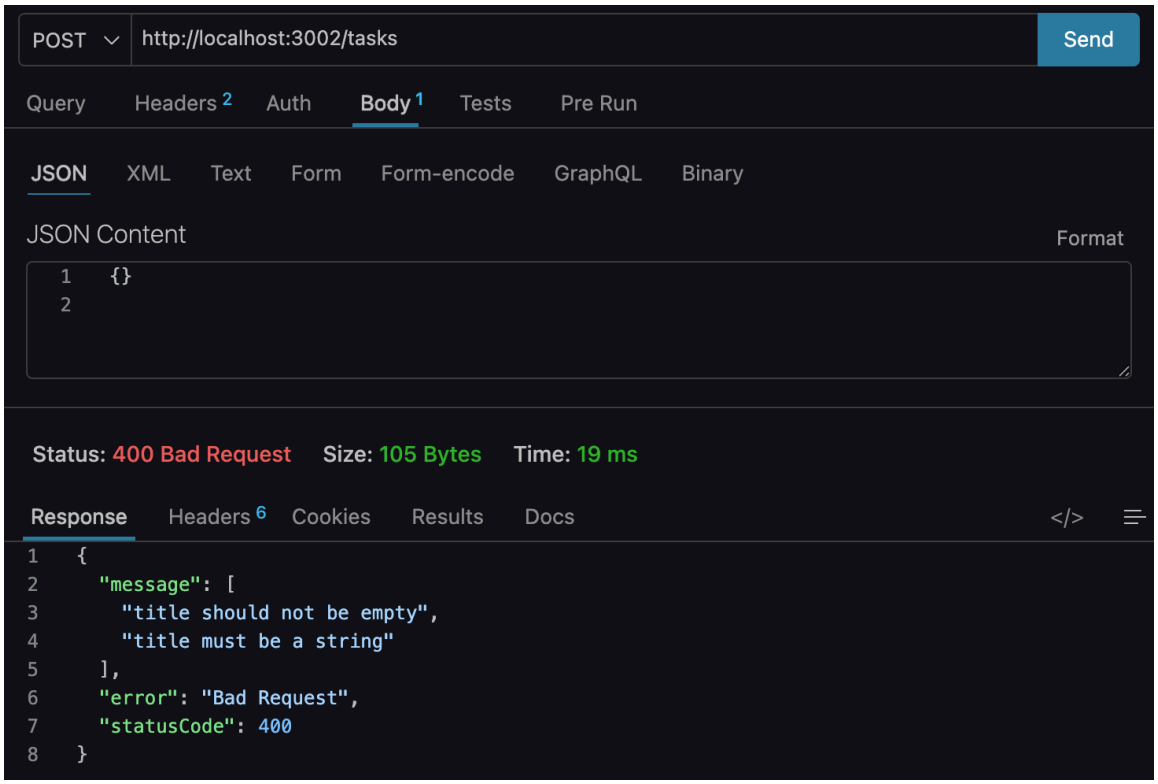
Post: (a) a valid title



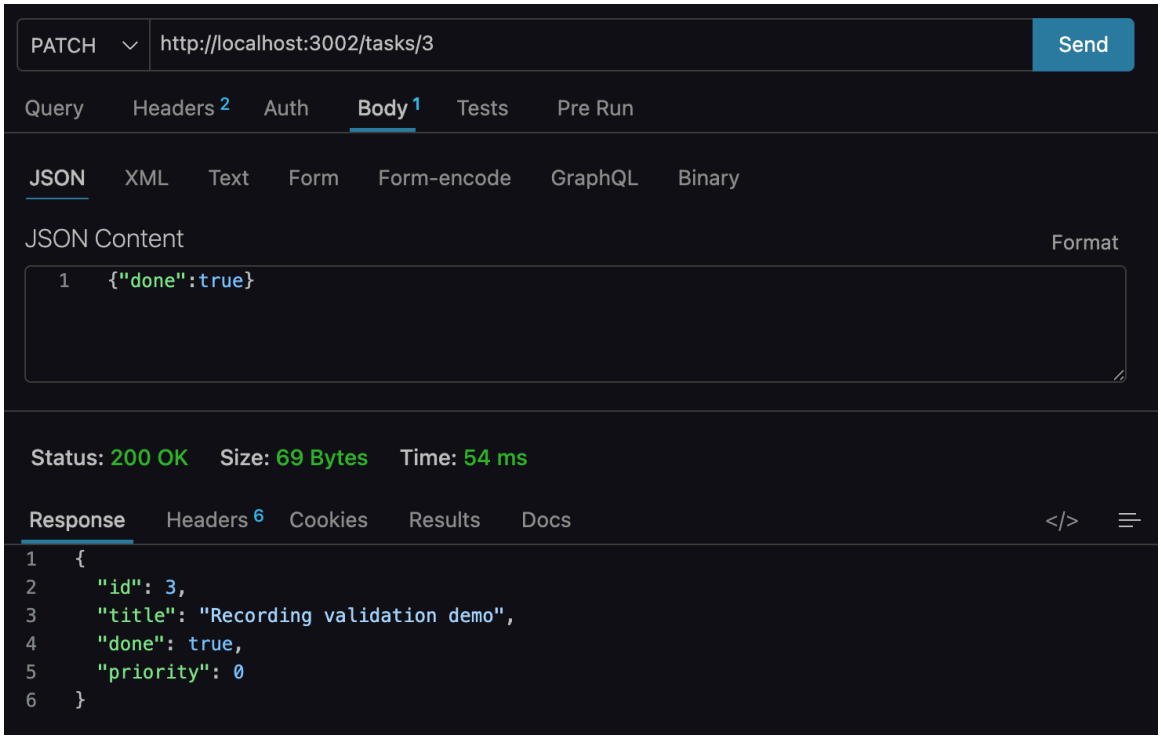
(b) an empty title



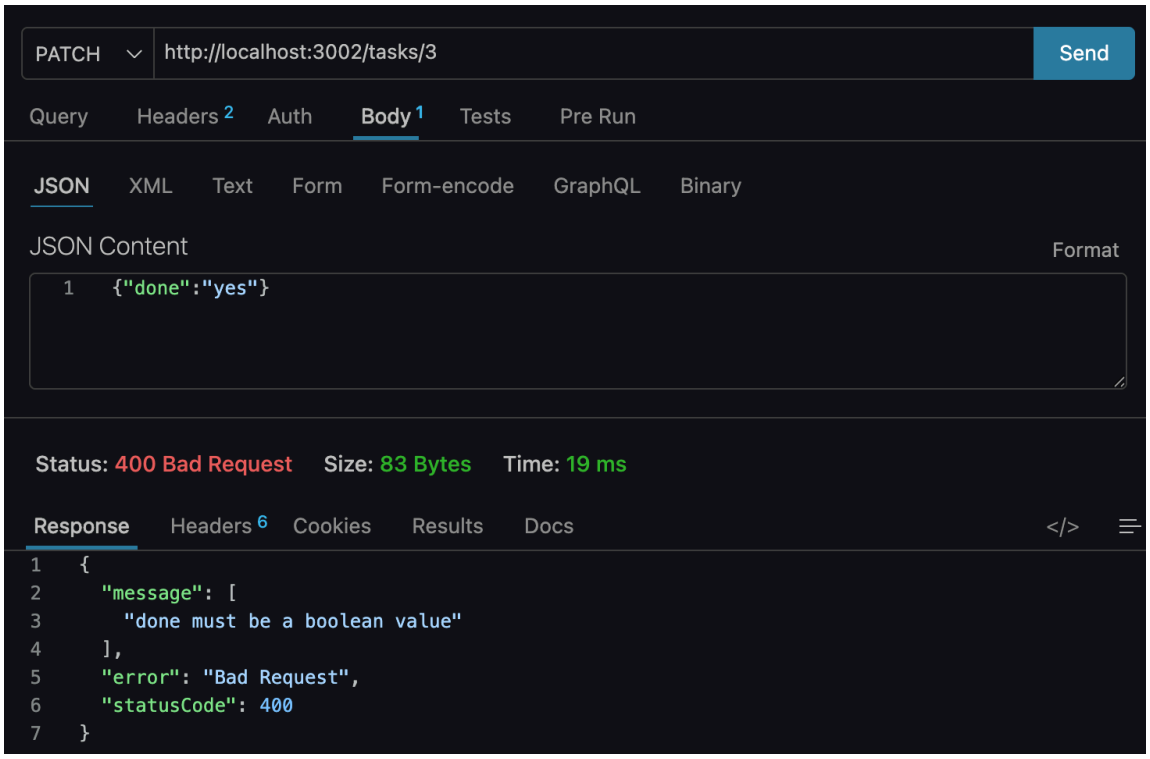
(c) a request with no title at all.



PATCH: (d) { "done": true }



(e) { "done": "yes" } (text instead of true or false)



(f) an empty body {}

